

Automating with Bash

NUnix · L3 MIAGE · CM3 · 90 minutes

ISTIC · University of Rennes 2026–2027



1/60

Writing Bash scripts



A script saves commands so you can run them again.
Arguments let you use the same script with different inputs.

1. Read arguments and store values in variables.
2. Use conditions, loops and functions.
3. Handle errors and test the script.

2/60

1. Scripts, arguments and variables



Run a script, pass arguments and store values

3/60

Bash: Bourne again



Bash = Bourne-Again SHell.

The name puns on **Stephen Bourne**, author of the earlier Unix shell `sh`.

We use Bash syntax in this course.
`sh script.sh` may select a different shell with fewer features.

4/60

A first script



```
#!/bin/bash
# hello.sh
name="MIAGE"
echo "Hello, $name"
```

- ▶ The first line selects an interpreter for direct execution.
- ▶ # begins a comment when used in this position.
- ▶ .sh is a naming convention, not a requirement.

5/60

Two ways to run a script



Invoke Bash

```
bash hello.sh
```

Bash reads the file.
It needs **read** access to it.

Execute the file

```
chmod u+x hello.sh
./hello.sh
```

Requires **execute** permission
and a usable interpreter.

6/60

The shebang: selecting an interpreter



```
#!/bin/bash
printf 'Hello, MIAGE\n'
```

On Linux, direct execution of `./hello.sh` uses the interpreter named after `#!`.

The extension `.sh` does not choose Bash.
The first line does. We call it the **shebang**.

7/60

One symbol, several names



Symbol	Names you may hear
#	hash, number sign; pound sign in US usage
#	French: croisillon , also carré
#	The musical sharp sign (<i>dièse</i>)

#!: **hash** followed by **bang**.
The symbol's name and its role depend on context.

8/60

Positional parameters



```
./hello.sh Alice "L3 MIAGE"
```

Parameter	Value inside this script
\$0	./hello.sh
\$1	Alice
\$2	L3 MIAGE
\$#	2
"\$@"	Every argument, each kept as a separate word

Passing arguments with spaces



```
set -- "L3 MIAGE" Unix
printf '<%s>\n' "$@"
printf '<%s>\n' "$*"

<L3 MIAGE>
<Unix>
<L3 MIAGE Unix>
```

Assigning and reading a variable



```
course="NUnix"  
echo "$course"  
course="Bash scripting"  
echo "Today: $course"
```

Assignment: name=value, with **no spaces** around =.
Expansion: \$name or \${name}.

11/60

Quoted variable expansions



```
filename="meeting notes.txt"  
cat "$filename"  
echo '${filename}'  
echo "${filename}"
```

- ▶ Double quotes preserve the expanded value as one argument.
- ▶ Single quotes preserve the literal text \${filename}.

12/60

Capturing command output



```
directory=$(pwd)
message="$1 and $2"
echo "$directory"
echo "$message"
```

`$(command)` captures the command's **standard output**.
\$1 and \$2 are positional arguments, introduced earlier.

13/60

Printing values



```
name="Alice"
echo "Hello, $name"
printf 'Hello, %s\n' "$name"
```

`echo` is convenient for simple messages.
`printf` gives explicit control over formatting and newlines.

14/60

Exporting a variable



```
message="hello"  
bash -c 'echo "$message"'  
export message  
bash -c 'echo "$message"'
```

The first child prints an empty line.
After export, the next child prints **hello**.

15/60

A child receives a copy



```
export message="parent"  
bash -c 'message="child"; echo "$message"'  
echo "$message"
```

```
child  
parent
```

16/60

2. Conditions and exit status



Choose which commands to run with if, && and ||

17/60

Tests are commands too



```
test -d logs  
[ -d logs ]  
[[ -d logs ]]  
(( 3 < 5 ))
```

- ▶ test and [are command names.] closes the [form.
- ▶ [[...]] is a Bash conditional construct.
- ▶ ((...)) tests an arithmetic expression.

18/60

if, elif and else



```
if [ "$count" -gt 10 ]; then
    echo "More than ten"
elif [ "$count" -eq 10 ]; then
    echo "Exactly ten"
else
    echo "Fewer than ten"
fi
```

The first successful test selects the branch.
else handles the remaining case.

19/60

The condition can be grep



```
if grep ' ERROR ' app.log; then
    echo "At least one error was found"
else
    echo "No match, or grep could not read the file"
fi
```

if checks a **command's status**.

Square brackets are needed only when we choose a test command.

20/60

Conditional command lists



```
mkdir results && echo "Directory created"
test -d results || echo "Directory is missing"
```

- ▶ a && b: run b only when a succeeds.
- ▶ a || b: run b only when a fails.
- ▶ ! a: invert the success/failure result.

21/60

Which messages appear?



```
true || echo "A"
false || echo "B"
true && echo "C"
false && echo "D"
```

```
B
C
```

22/60

Checkpoint: command status



- | | |
|----------------------------------|--|
| 1. Status 0: true or false? | 1. True / success. |
| 2. Does false; echo yes print? | 2. Yes. ; always continues. |
| 3. Does false && echo yes print? | 3. No. |
| 4. Status of false true? | 4. 0, from the final executed command, true. |

23/60

Check the argument count



Save as count-lines.sh:

```
#!/bin/bash
if [ "$#" -ne 1 ]; then
    printf 'Usage: %s FILE\n' "$0" >&2
    exit 2
fi
wc -l < "$1"
```

Check the argument count **before** using \$1.
Print the usage message on **stderr**.

24/60

Integer arithmetic



```
count=3
echo "$((count + 2))"
count=$((count + 1))
((count += 2))
echo "$count"
```

- ▶ `$((...))` expands to an integer result.
- ▶ `((...))` is a Bash arithmetic command.

25/60

3. Loops



Process a list of arguments or read a file line by line

26/60

for visits a list of words



```
for group in L3info L3miage M1info; do
    echo "$group"
done
```

Each iteration assigns the next word to group, then runs the body.

The list can contain literal words, numbers, filenames or "\$@".

27/60

while repeats after a successful test



```
i=1
while [ "$i" -lt 10 ]; do
    echo "$i"
    ((i += 1))
done
```

Test **before** each iteration.

The loop prints the integers **1 through 9**.

28/60

while can test a whole command list



```
i=1
while
  echo "Checking i=$i"
  [ "$i" -le 3 ]
do
  echo "Body: $i"
  ((i += 1))
done
```

The **last command** before do decides whether the body runs.

29/60

The condition runs once more than the body



```
Checking i=1
Body: 1
Checking i=2
Body: 2
Checking i=3
Body: 3
Checking i=4
```

- ▶ At **1, 2, 3**, the test succeeds.
- ▶ At **4**, the final test fails.
- ▶ The body does not run a fourth time.

30/60

Read a file line by line



```
while IFS= read -r name; do
    printf 'Hello, %s\n' "$name"
done < names.txt
```

- ▶ read succeeds when it reads a complete line.
- ▶ At **EOF**, it fails and the loop stops.
- ▶ < feeds the file to the **whole loop**.

31/60

until repeats after a failed test



```
i=1
until [ "$i" -ge 10 ]; do
    echo "$i"
    ((i += 1))
done
```

This also prints **1 through 9**.
The loop stops when the condition succeeds.

32/60

break, continue and exit



Command	Effect
continue	Skip the rest of the current iteration
break	Leave the current loop
exit n	End the script with status n

After break, execution continues **after the loop**.

33/60

case selects a matching pattern



```
read -r -p "Enter a command: " choice
case "$choice" in
  all|ALL)
    ls -la ;;
  list|LIST)
    ls -l ;;
  *)
    echo "Invalid choice" ;;
esac
```

| separates alternative patterns. ;; ends a branch.

34/60

4. Functions and debugging



Group commands in functions and trace script execution

35/60

Defining and calling a function



```
greet() {  
    echo "Hello, $1"  
}  
  
greet "MIAGE"
```

- ▶ Define the function **before calling it**.
- ▶ Inside it, \$1 is its first argument.
- ▶ `return n` finishes the function with exit status n.

36/60

Check syntax and trace execution



```
bash -n count-lines.sh
bash -x count-lines.sh notes.txt
```

- ▶ -n: parse the script without executing its commands.
- ▶ -x: execute and trace expanded commands on stderr.
- ▶ Check output **and** status on ordinary and failing inputs.

37/60

CM3 summary: Bash scripts



1. Define inputs, outputs and failure statuses.
2. Quote variable and filename arguments.
3. Use command status for branches and loop conditions.
4. Test normal, empty and failing inputs.

38/60

Optional reference



More examples and a command reference

39/60

Command substitution



```
echo "Current directory: $(pwd)"
```

The shell runs `pwd` and replaces `$(pwd)` with its **standard output**.

The older form is ``pwd``.

We will write `$(pwd)` because nested commands are easier to read.

40/60

Newlines, semicolons and continuations



```
echo "first"
echo "second"; echo "third"
echo "a longer message" \
    "continued on the next line"
```

- ▶ A newline or ; separates commands.
- ▶ A backslash immediately before a newline continues the line.

41/60

Reading a line



```
read -r -p "Your name: " name
echo "Hello, $name"
```

- ▶ read reads from **standard input**.
- ▶ -p displays a prompt in an interactive terminal.
- ▶ -r preserves backslashes in the input.

42/60

Integer, readonly and array variables



```
declare -i count=3
readonly course="NUnix"
declare -a words
words[0]="one"
words[1]="two"
echo "${words[0]}"
```

unset count removes a variable. A readonly variable cannot be reassigned or unset in that shell.

Familiar environment variables



Variable	Meaning
HOME	Home directory
USER	Account name, usually set at login
PWD, OLDPWD	Current and previous working directories
PATH	Colon-separated command search directories
SHELL	Configured login shell, not necessarily the current shell
TERM	Terminal type
HOSTNAME	Host name in Bash
MANPATH	Manual search path, when explicitly set

Arithmetic with let and expr



```
let 'count = 3 + 5'
echo "$count"
expr 3 + 5
expr 3 \* 5
```

- ▶ let evaluates an arithmetic expression in Bash.
- ▶ expr takes separate arguments, so its spaces matter.
- ▶ Escape * so the shell passes it to expr literally.

45/60

Comparing numbers and strings



Relation	Integers with [	Strings with [
Equal	-eq	=
Different	-ne	!=
Less / greater	-lt, -gt	\<, \>
Less / greater or equal	-le, -ge	No dedicated operator

```
[ "$count" -gt 3 ]
[ "$name" = "Alice" ]
```

46/60

Testing strings and files



Test	True when...
-n "\$value"	The string is nonempty
-z "\$value"	The string is empty
-e "\$path"	The path exists
-f "\$path"	It is a regular file
-d "\$path"	It is a directory
-s "\$path"	Its size is greater than zero
-r, -w, -x	Read, write or execute/search access is allowed

47/60

Checkpoint: arithmetic and tests



- | | |
|--|-------------------------------------|
| 1. Value of $((5 / 2))$? | 1. 2 (integer division). |
| 2. Is <code>n = 3</code> an assignment? | 2. No. Use <code>n=3</code> . |
| 3. Status of <code>false</code> ? | 3. 1 |
| 4. Does <code>-f</code> accept every existing directory? | 4. No. It tests for a regular file. |

48/60

Checking access to a file



```
#!/bin/bash
read -r -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
    echo "File is not writable"
    exit 1
elif [ ! -r "$filename" ]; then
    echo "File is not readable"
    exit 1
else
    echo "File is writable and readable"
fi
```

49/60

Comparing two files



```
diff "$1" "$2" > diff.out
status=$?
if [ "$status" -eq 0 ]; then
    echo "Files are identical"
elif [ "$status" -eq 1 ]; then
    cat diff.out
else
    echo "Comparison failed" >&2
fi
rm -f diff.out
```

50/60

Copying a known list of files



```
#!/bin/bash
destination=$1
for file in f1 f2 f3; do
    cp "$file" "$destination/$file"
done
```

Call with an **existing destination directory** as the first argument.

51/60

A loop can feed a pipeline



```
for number in 6 3 1 2; do
    echo "$number"
done | sort -n
```

```
1
2
3
6
```

52/60

Repeating until interrupted



```
while true; do
  echo "alive..."
  sleep 3
done
```

true always succeeds.
Use **Ctrl+C** to interrupt this foreground loop.

53/60

Skipping 4, stopping at 7



```
for x in 1 2 3 4 5 6 7 8 9; do
  if [ "$x" -eq 4 ]; then
    continue
  fi
  echo "$x"
  if [ "$x" -eq 7 ]; then
    break
  fi
done
echo "After the loop: x=$x"
```

54/60

The loop's output



```
1
2
3
5
6
7
After the loop: x=7
```

- ▶ At **4**, continue skips echo.
- ▶ At **7**, echo runs before break.
- ▶ The final message runs after the loop.

55/60

Function scope



```
outside="a global variable"
mine() {
    local inside="this is local"
    echo "$outside"
    echo "$inside"
    outside="a global with new value"
}
echo "$outside"
mine
echo "$outside"
echo "${inside-unset}"
```

56/60

What remains after the call



```
a global variable
a global variable
this is local
a global with new value
unset
```

outside changed.
The function's **local variable** is no longer in scope.

Builtins: shell and process control



Builtin	Use
cd, pwd	Change / report the current directory
history	Display command history
exec	Replace the shell with another program
exit	End the current shell
kill	Send a signal
trap	Arrange a response to a signal or shell event

Builtins: data and parameters



Builtin	Use
declare, readonly, local	Define variable attributes or scope
export, set	Manage the environment / shell settings
echo, read	Print output / read input
let, test	Arithmetic / condition tests
shift	Discard leading positional parameters
return	End a function and report its status

Reading a small script



```
#!/bin/bash
for filename in "$@"; do
  if [ -f "$filename" ]; then
    wc -l "$filename"
  fi
done
```

For each argument that names a regular file, print its line count.